Absolutely. Here are your **React Day 3 complete interview notes**, focused on what you learned today and phrased so you can revise them before placements.

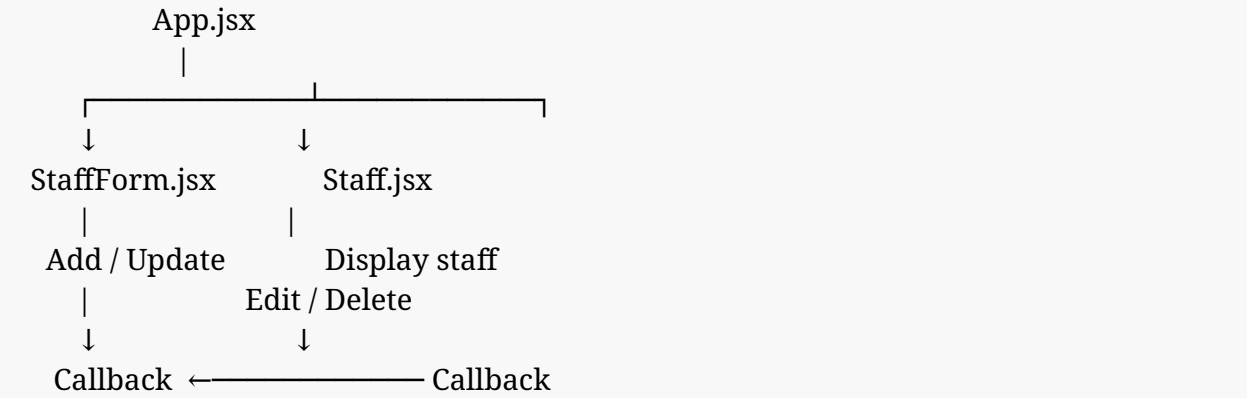
📘 React Day 3 — Complete Interview Notes

1. Main Goal of Day 3

Today we improved our React CRUD application by making it:

- cleaner
- reusable
- easier to maintain
- better at handling errors
- better at handling Add/Edit modes
- better structured between parent and child components

Our architecture is:



---

2. fetchStaff() Function

Instead of writing the API request directly inside useEffect(), we created a separate function:

```
async function fetchStaff() {
  // API logic
}
```

Why?

Because it keeps the API logic separate and reusable.

```
fetchStaff()
   ↓
GET /api/staff
   ↓
Spring Boot
   ↓
MySQL
   ↓
JSON response
   ↓
setStaffList(data)
```

Interview answer

Why did you create a separate fetchStaff() function?

Answer:

> "I created a separate fetchStaff() function to keep the API-fetching logic reusable and organized. useEffect() simply calls this function when the component loads."

---

3. async and await

API requests take some time because the browser communicates with the backend.

We use:

```
async function fetchStaff() {
```

and:

```
const response = await fetch(...);
```

async

Indicates that the function performs asynchronous work.

await

Waits for the asynchronous operation to complete.

Simple flow:

```
React
  ↓
API request
  ↓
WAIT
  ↓
Backend response
  ↓
Continue execution
```

Interview answer

> "async allows a function to work with asynchronous operations, while await waits for the Promise to complete before continuing."

---

4. try / catch / finally

We use these for error handling.

```
try {
   // API request
}
catch (error) {
   // handle error
}
finally {
   // always execute
}
```

try

Contains the code we want to attempt.

catch

Runs when an error occurs.

finally

Runs whether the operation succeeds or fails.

For example:

```
try {
  const response = await fetch(url);
}
catch (error) {
  console.error(error);
}
finally {
  setLoading(false);
}
```

Interview answer

> "try contains the operation, catch handles errors, and finally executes regardless of whether the operation succeeds or fails."

---

5. Loading State

We use:

```
const [loading, setLoading] = useState(true);
```

When API work starts:

```
setLoading(true);
```

When it finishes:

```
setLoading(false);
```

Usually we put the final change in:

```
finally {
  setLoading(false);
}
```

because loading should stop even if the API fails.

UI

```
{loading && <p>Loading staff...</p>}
```

Meaning:

If loading is true, show “Loading staff...”

---

6. Error State

We created:

```
const [error, setError] = useState("");
```

When an API fails:

```
setError("Failed to load staff");
```

Then:

```
{error && <p>{error}</p>}
```

Meaning:

If an error message exists, display it.

---

7. response.ok

After calling:

```
const response = await fetch(url);
```

we check:

```
if (!response.ok) {
   throw new Error("Failed to fetch staff");
}
```

response.ok is a Boolean.

```
Successful response
      ↓
response.ok = true

Unsuccessful response
```

```
    ↓
response.ok = false
```

Important interview point

fetch() does **not automatically throw an error just because the HTTP response is 404 or 500**.

Therefore we explicitly check:

```
if (!response.ok)
```

and throw an error ourselves.

Interview answer

> "response.ok tells us whether the HTTP response was successful. If it is false, I throw an error so that the catch block can handle the failure."

---

8. response.json()

After a successful API request:

```
const data = await response.json();
```

This converts the JSON response into a JavaScript object/array that React can work with.

Example backend response:

```
[
 {
  "id": 1,
  "name": "Aniket",
  "department": "IT",
  "role": "Developer"
 }
]
```

Then:

```
setStaffList(data);
```

stores it in React state.

---

9. Edit Flow — VERY IMPORTANT ⭐

This is one of the most important concepts from today.

Suppose we have:

```
Aniket
IT
Developer

[Edit]
```

When we click Edit:

```
1. User clicks Edit

        ↓

2. Staff.jsx calls onEdit(id)

        ↓

3. App.jsx receives the ID

        ↓

4. App uses find()

        ↓

5. Selected staff is found

        ↓

6. setEditingStaff(staff)

        ↓

7. StaffForm receives editingStaff
```

```
      ↓
8. useEffect() detects change
      ↓
9. setFormData(...)
      ↓
10. Form gets populated
```

The key chain to remember:

```
Edit
 ↓
onEdit(id)
 ↓
find()
 ↓
setEditingStaff()
 ↓
useEffect()
 ↓
setFormData()
 ↓
Form populated
```

---

10. find()

We use:

```
const staff = staffList.find(
   (staff) => staff.id === id
);
```

find() searches the array and returns the matching staff object.

Example:

```
staffList:

Aniket → ID 1
Rahul  → ID 2
Priya  → ID 3
```

If:

```
id = 2
```

then:

```
find()
```

returns Rahul.

Interview answer

> “find() is used to locate the specific staff object whose ID matches the ID received from the Edit button.”

---

11. editingStaff State

We created:

```
const [editingStaff, setEditingStaff] = useState(null);
```

Initially:

```
editingStaff = null
```

means:

> We aren’t editing anyone.

After clicking Edit:

```
setEditingStaff(staff);
```

Now:

```
editingStaff = selected staff
```

means:

> We are editing this staff member.

---

12. useEffect() for Editing

In StaffForm.jsx:

```
useEffect(() => {

  if (editingStaff) {

    setFormData({
      name: editingStaff.name,
      department: editingStaff.department,
      role: editingStaff.role
    });

  }

}, [editingStaff]);
```

The important part:

```
[editingStaff]
```

means the effect watches editingStaff.

When it changes:

```
editingStaff changes
      ↓
useEffect runs
      ↓
formData changes
      ↓
form gets populated
```

---

13. Controlled Components

Our form inputs are controlled by React state.

Example:

```
<input
  name="name"
  value={formData.name}
  onChange={handleChange}
/>
```

The input's value comes from:

```
formData.name
```

and changes through:

```
handleChange()
```

So:

```
User types
  ↓
onChange
  ↓
handleChange
  ↓
setFormData
  ↓
React state changes
  ↓
Input updates
```

---

14. handleChange()

We use:

```
function handleChange(e) {

  const { name, value } = e.target;

  setFormData({
    ...formData,
    [name]: value
  });
}
```

e.target

Represents the input that triggered the event.

e.target.value

Contains what the user typed.

e.target.name

Contains the input's name.

For example:

```
<input name="department" />
```

Then:

```
name = "department"
```

and:

```
value = "IT"
```

---

15. Dynamic Property [name]

This is an important JavaScript concept.

We have:

```
[name]: value
```

Suppose:

```
name = "department"
value = "IT"
```

Then JavaScript effectively creates:

```
department: "IT"
```

If:

```
name = "role"
value = "Developer"
```

then:

```
role: "Developer"
```

This lets one handleChange() function handle multiple inputs.

---

16. Spread Operator

We use:

```
setFormData({
   ...formData,
   [name]: value
});
```

The:

```
...formData
```

copies the existing fields.

Then:

```
[name]: value
```

updates the field that changed.

Example:

```
Before:


{
 name: "Aniket",
 department: "IT",
 role: "Developer"
}
```

User changes role:

```
Manager
```

Result:

```
{
 name: "Aniket",
 department: "IT",
```

```
 role: "Manager"
}
```

---

17. Form Validation

Before sending data to the backend, we check:

```
if (!formData.name.trim()) {
   newErrors.name = "Name is required";
}
```

Similarly for department and role.

Then:

```
if (Object.keys(newErrors).length > 0) {
   setErrors(newErrors);
   return;
}
```

Meaning:

> If there are validation errors, stop the function and don't call the API.

Flow:

```
Submit
 ↓
Validate
 ↓
Errors?
 ├── YES → show errors → STOP
 └── NO  → API request
```

---

18. One Form for Add and Update

This is one of today's most important design improvements.

We don't need:

```
AddStaffForm
UpdateStaffForm
```

Instead:

```
StaffForm
```

handles both.

We determine the mode using:

```
if (editingStaff)
```

Add mode

```
editingStaff = null
      ↓
POST
      ↓
Create staff
```

Update mode

```
editingStaff exists
      ↓
PUT
      ↓
Update staff
```

---

19. POST vs PUT

POST

Used when creating new staff:

```
POST /api/staff
```

PUT

Used when updating existing staff:

```
PUT /api/staff/{id}
```

For example:

```
PUT /api/staff/5
```

means:

> Update staff whose ID is 5.

---

20. JSON.stringify()

When sending form data:

```
body: JSON.stringify(formData)
```

We convert the JavaScript object into JSON.

For example:

```
{
  name: "Aniket",
  department: "IT",
  role: "Developer"
}
```

becomes JSON data that can be sent to Spring Boot.

We also use:

```
headers: {
  "Content-Type": "application/json"
}
```

to tell the backend:

> “The request body contains JSON.”

---

21. Child → Parent Communication

This is a major React interview topic.

Our structure:

```
App.jsx
  ↓
StaffForm.jsx
```

App passes functions:

```
<StaffForm
  onStaffAdded={handleStaffAdded}
  onStaffUpdated={handleStaffUpdated}
  ...
/>
```

Then StaffForm calls:

```
onStaffUpdated(data);
```

The flow is:

```
StaffForm
  ↓
callback
  ↓
App
  ↓
setStaffList()
```

Interview answer

> “React data normally flows from parent to child through props. For child-to-parent communication, we pass a callback function from the parent and call it from the child.”

---

22. Why Doesn’t `StaffForm` Directly Modify `staffList`?

Because:

```
staffList
```

belongs to App.jsx.

The child should communicate the result to the parent.

```
StaffForm
  ↓
onStaffUpdated(data)
  ↓
App.jsx
```

```
   ↓
setStaffList()
```

This keeps component responsibilities clear.

---

23. Updating Array with map()

After updating a staff member:

```
setStaffList(
   staffList.map(
      (staff) =>
         staff.id === updatedStaff.id
            ? updatedStaff
            : staff
   )
);
```

Think:

```
Old list:

Aniket
Rahul
Priya
```

Updating Rahul:

```
Aniket → keep
Rahul  → replace
Priya  → keep
```

Result:

```
Aniket
Updated Rahul
Priya
```

Remember:

```
map() → create a new array with transformed/updated values
```

---

24. Delete with filter()

For deletion:

```
setStaffList(
  staffList.filter(
    (staff) => staff.id !== id
  )
);
```

Suppose:

```
Aniket
Rahul
Priya
```

Delete Rahul:

```
Aniket → keep
Rahul  → remove
Priya  → keep
```

Result:

```
Aniket
Priya
```

Remember:

filter() → create a new array excluding unwanted items

---

25. map() VS filter() VS find()

⭐ **Memorize this table.**

Method	Purpose
find()	Find one item
map()	Transform/update items or render a list
filter()	Remove/exclude items

Example:

```
find()   → Find Rahul
map()    → Replace Rahul
filter() → Remove Rahul
```

---

26. Cancel Edit

When the user clicks Cancel:

```
function handleCancelEdit() {
   setEditingStaff(null);
}
```

This means:

```
We are no longer editing anyone.
```

Then:

```
editingStaff = null
      ↓
useEffect()
      ↓
formData reset
      ↓
Form returns to Add mode
```

The important point:

> **Cancel does not send a PUT request to the backend.**

---

27. Add Flow

Know this complete flow for interviews:

```
User enters staff details
      ↓
Controlled inputs update formData
      ↓
User clicks Add Staff
      ↓
Validation
```

```
      ↓
POST /api/staff
      ↓
Spring Boot
      ↓
Database
      ↓
New staff returned
      ↓
onStaffAdded(data)
      ↓
App updates staffList
      ↓
React re-renders
```

---

28. Update Flow

```
User clicks Edit
      ↓
onEdit(id)
      ↓
find(id)
      ↓
setEditingStaff(staff)
      ↓
useEffect()
      ↓
formData populated
      ↓
User changes data
      ↓
Click Update
      ↓
Validation
      ↓
PUT /api/staff/{id}
      ↓
Spring Boot
      ↓
Database
      ↓
```

```
Updated staff returned
        ↓
onStaffUpdated(data)
        ↓
App uses map()
        ↓
staffList updated
        ↓
React re-renders
```

---

29. Delete Flow

```
User clicks Delete
        ↓
onDelete(id)
        ↓
DELETE /api/staff/{id}
        ↓
Spring Boot
        ↓
Database
        ↓
Success
        ↓
filter()
        ↓
staffList updated
        ↓
React re-renders
```

---

30. Error Debugging — 404

Suppose:

```
PUT /api/staff/5
```

returns:

```
404 Not Found
```

First 3 things to check:

1. URL

Check:

```
http://localhost:8080/api/staff/5
```

Does it match the backend endpoint?

2. ID

Check:

```
editingStaff.id
```

Is the correct ID being sent?

3. Backend endpoint/data

Check the Spring Boot controller:

```
@PutMapping("/{id}")
```

and verify that the staff with that ID exists.

Interview answer

> "For a 404 during an update, I would first verify the frontend URL, then verify the ID being sent, and finally check the backend endpoint mapping and whether the record exists."

⭐ Top 15 Interview Questions From Day 3

Practice these until you can answer without looking.

Q1

Why use a separate `fetchStaff()` function?

Q2

What is the purpose of `async/await`?

Q3

Explain try/catch/finally.

Q4

What does response.ok mean?

Q5

Why do we use response.json()?

Q6

Explain the complete Edit flow.

Q7

What is editingStaff?

Q8

Why do we use useEffect([editingStaff])?

Q9

What is a controlled component?

Q10

What does e.target.value represent?

Q11

Why use [name]: value?

Q12

Why can one form handle both POST and PUT?

Q13

Why use map() for updating and filter() for deleting?

Q14

Why does the child use callback props to communicate with the parent?

Q15

What would you check if an update API returns 404?

🏆 30-Second Interview Answer

If an interviewer asks:

> **“Explain your React staff management application.”**

You can answer:

> “I built a React-based staff management application with CRUD functionality. App.jsx manages the main staff list and communicates with the backend API. StaffForm is a reusable controlled form that handles both adding and updating staff using POST and PUT requests. Staff displays individual staff members and provides Edit and Delete actions. For editing, I use find() to locate the selected staff, store it in editingStaff, and use useEffect() to populate the form. After an update, I use map() to replace the updated staff in the state, while filter() is used to remove a staff member after deletion. I also added validation, loading states, and error handling using try/catch/finally.”

That’s a **very solid placement-level explanation** for what you’ve learned so far.

Your Day 3 React checklist

✅ async / await
✅ fetchStaff()
✅ try / catch / finally
✅ response.ok
✅ loading state
✅ error state
✅ find()
✅ editingStaff
✅ useEffect for editing
✅ controlled inputs
✅ handleChange
✅ e.target
✅ spread operator
✅ dynamic [name]: value
✅ validation

```
✅ POST
✅ PUT
✅ JSON.stringify()
✅ callback props
✅ child → parent communication
✅ map() for update
✅ filter() for delete
✅ Cancel edit
✅ 404 debugging
✅ Complete CRUD flow
```

React Day 3 is now complete. 🚀